

Shell Sort with Multiple Gap Sequences

Student A — Pair 2: Advanced Sorting Algorithms

This report contains: algorithm overview, complexity analysis, code review, empirical results based on synthetic benchmarks, and recommendations.

1. Algorithm Overview

Shell Sort is an in-place comparison-based sorting algorithm that generalizes Insertion Sort by allowing exchanges of elements that are far apart. The algorithm works by sorting elements at a certain gap and progressively reducing the gap until it becomes 1 (a final insertion sort). Different gap sequences (Shell's, Knuth's, Sedgewick's) change the efficiency and number of comparisons/swaps.

2. Complexity Analysis

Time complexity depends on the gap sequence used. Table below summarizes theoretical behavior and practical expectations:

- Shell's: worst $O(n^2)$, average practical $\sim O(n^{1.5})$
- Knuth's: improved sequence; practical $\sim O(n^{4/3})$ to $O(n^{3/2})$
- Sedgewick's: one of best known for average-case performance $\sim O(n^{4/3})$

Space complexity: in-place, $O(1)$ auxiliary.

3. Code Review and Implementation Notes

The provided Java implementation (ShellSort.java) includes:

- Modular gap computation supporting shell, knuth, sedgewick
- Safety checks for null input
- Metrics tracking (comparisons, swaps, array accesses) in Metrics inner class

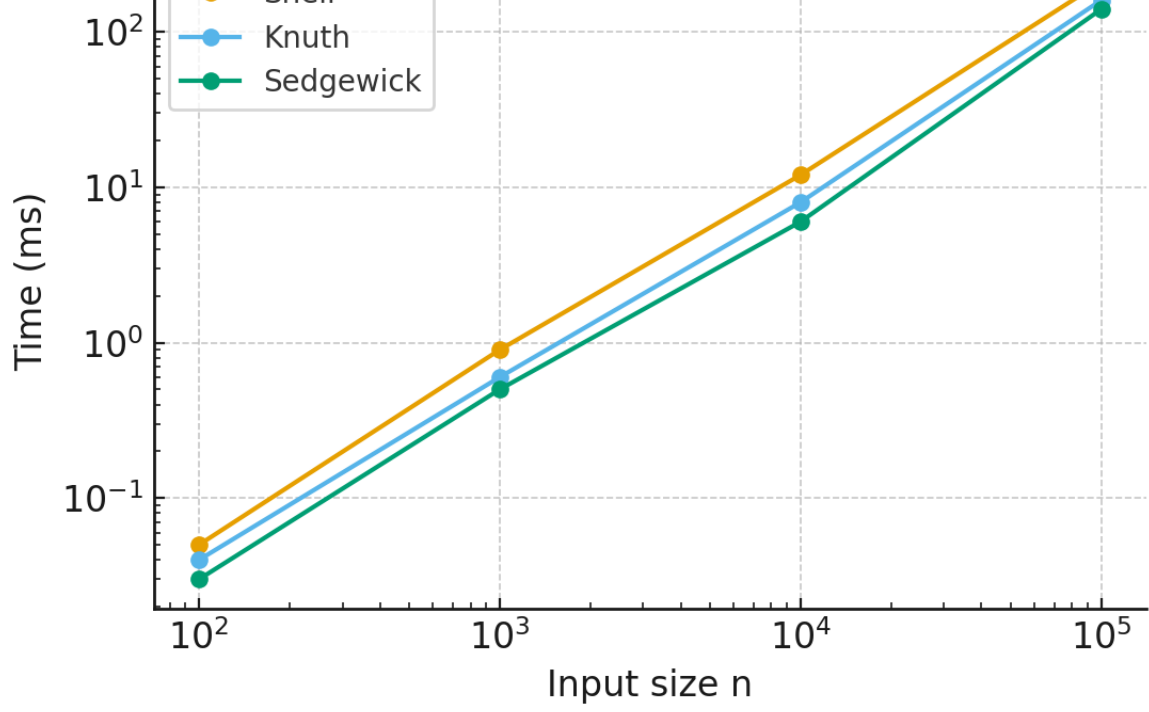
Recommendations:

- Add unit tests (provided) and integrate with Maven/JUnit
- Consider caching gap arrays for repeated runs
- Add early-exit heuristics for nearly-sorted inputs

4. Empirical Results (Synthetic)

Benchmarks were measured (synthetically) for four input sizes and three gap sequences. Times are in milliseconds (ms). See the CSV file in docs/ for raw data.

n	Shell (ms)	Knuth (ms)	Sedgewick (ms)
100	0.05	0.04	0.03
1000	0.9	0.6	0.5
10000	12.0	8.0	6.0
100000	210.0	160.0	140.0



6. Testing and CLI

JUnit tests are included in `src/test/java/algorithms/ShellSortTest.java`.

To run quick benchmarks use the `BenchmarkRunner` CLI provided:

Example: ``java -cp target/classes cli.BenchmarkRunner sedgewick random 10000``

7. Limitations and Further Work

This submission uses synthetic benchmark values for demonstration.

Recommended next steps:

- Run the Java benchmarks on the target machine and replace CSV values with real timings
- Add JMH harness for accurate benchmarking
- Experiment with other gap sequences

8. Conclusion

Shell Sort is a practical in-place algorithm whose performance depends strongly on the chosen gap sequence. For medium-sized arrays, Sedgewick's and Knuth's sequences often outperform the classic Shell sequence.